Service-oriented Software Engineering (SoSE)

(must have) Advanced Software Engineering ingredients towards Service Orientation

(1/3)

Marco Autili

University of L'Aquila

marco.autili@univaq.it

http://people.disim.univaq.it/marco.autili/

About these slides

- These slides are mostly based on the original slides (publicly available from the official sites of the two books that I will adopt as reference textbooks for this course)

- The complete responsibility of the changes made by myself is addressable to me

Topics covered by these slides

Software reuse

- Reuse-based Software Engineering
- The reuse landscape
- Implementing software reuse

Component-based software engineering

- Components and component models
- CBSE processes
- Component composition

Distributed software engineering

- Distributed systems issues
- Client–server computing
- Architectural patterns for distributed systems
- Software as a service

WHY these topics?

Software reuse

- SOA was born with the word reuse in mind
- Services can be reused (development with reuse)
- Services should be developed so as to be reusable (development for reuse)

Component-based Software Engineering

- SoSE shares many similarities with CBSE
- SoSE overcomes the obstacles that hindered the uptake of CBSE

Distributed Software Engineering

- By nature, any SOA-based system is (should be engineered as, and should be realized as) a distributed system

Topics covered by these slides

Software reuse

- Reuse-based Software Engineering
- The reuse landscape
- Implementing software reuse

Component-based software engineering

- Components and component models
- CBSE processes
- Component composition

Distributed software engineering

- Distributed systems issues
- Client–server computing
- Architectural patterns for distributed systems
- Software as a service

Software reuse

- In the past, software engineering was more focused on original development but …
- … it is now recognised that to achieve better software, more quickly and at lower cost, we need a design process that is based on systematic software reuse.
- Today, in most engineering disciplines, systems are designed by composing existing software.

Software reuse

- There has been a major switch to reuse-based development over the past 15 years.

- Indeed, over the past 25 years, many techniques have been developed to support software reuse, notably Component-based Software Engineering (CBSE)

- The move to reuse-based development has been in response to demands for
  - lower software production costs
  - lower maintenance cost
  - faster delivery of systems
  - increased software quality

Software reuse

Nowadays, there are many pieces of software available that can be tailored and adapted to specific customer needs

- More and more companies see their software as a valuable asset and promote reuse to increase their Return on software Investments
- Also, the open source movement has meant that there is a huge reusable code base available at low cost, "for free". This may be in the form of program libraries or entire applications.
- (non-proprietary) standards, such as Web Service standards, have made it easier to develop general services and reuse them across a range of applications.

Reuse-based Software Engineering

Software units that can be reused may be of radically different sizes

System reuse

- Complete systems, which may include several application programs may be reused.

Application reuse

- An application may be reused either by incorporating it without change into other or by developing application families.

Service reuse 

- Standards, such as Web Service standards, have made it easier to develop general services and reuse them across a range of applications.

Component reuse

- Components of an application from sub-systems to single objects may be reused.

Object and function reuse

- Small-scale software components that implement a single well-defined object or function may be reused. This form of reuse, based around standard libraries, has been common for the past 40 years.

Concept reuse

- A complementary form of reuse is "concept reuse" where, rather than reuse a software component, you reuse an idea, a way, or working or an algorithm.

- The concept that you reuse is represented in an abstract notation (e.g., a system model), which does not include implementation detail. It can, therefore, be configured and adapted for a range of situations.

- Concept reuse can be embodied in approaches such as design patterns (covered in Chapter 7), configurable system products, and program generators.

- When concepts are reused, the reuse process includes an activity where the abstract concepts are instantiated to create executable reusable components

Benefits of software reuse

Benefit	Explanation
Accelerated development	Bringing a system to market as early as possible is often more important than overall development costs. Reusing software can speed up system production because both development and validation time may be reduced.
Effective use of specialists	Instead of doing the same work over and over again, application specialists can develop reusable software that encapsulates their knowledge.
Increased dependability	Reused software, which has been tried and tested in working systems, should be more dependable than new software. Its design and implementation faults should have been found and fixed.

Benefits of software reuse

Benefit	Explanation
Lower development costs	Development costs are proportional to the size of the software being developed. Reusing software means that fewer lines of code have to be written.
Reduced process risk	The cost of existing software is already known, whereas the costs of development are always a matter of judgment. This is an important factor for project management because it reduces the margin of error in project cost estimation. This is particularly true when relatively large software components such as subsystems are reused.
Standards compliance	Some standards, such as user interface standards, can be implemented as a set of reusable components. For example, if menus in a user interface are implemented using reusable components, all applications present the same menu formats to users. The use of standard user interfaces improves dependability because users make fewer mistakes when presented with a familiar interface.

Problems with reuse

Problem	Explanation
Creating, maintaining, and using a component library	Populating a reusable component library and ensuring that software developers can use this library can be expensive. Development processes have to be adapted to ensure that the library is (re-)used.
Finding, understanding, and adapting reusable components	Software components have to be discovered in a library, understood and, sometimes, adapted to work in a new environment. Engineers must be reasonably confident of finding a component in the library before they include a component search as part of their normal development process.
Increased maintenance costs	If the source code of a reused software system or component is not available then maintenance costs may be higher because the reused elements of the system may become increasingly incompatible with system changes.

Problems with reuse

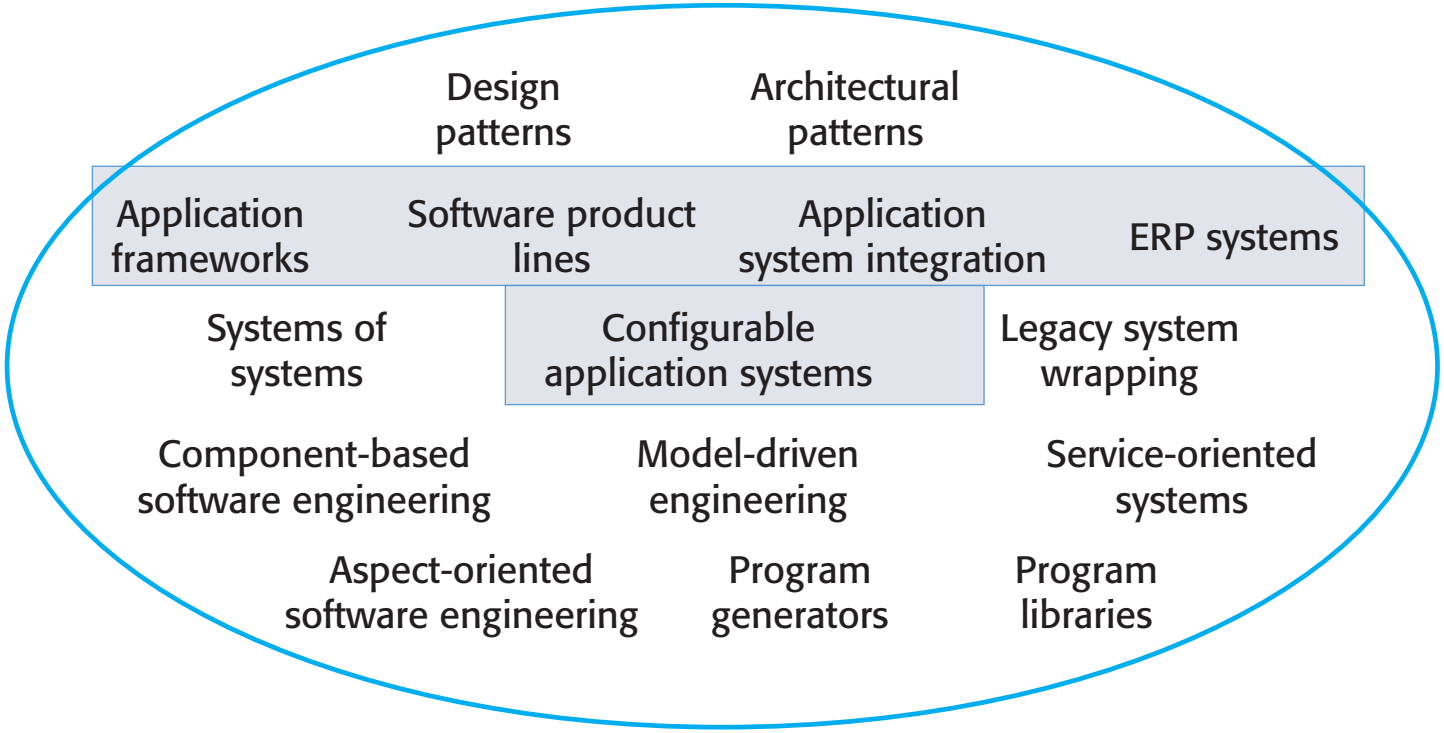
Problem	Explanation
Lack of tool support	Some software tools do not support development with reuse. It may be difficult or impossible to integrate these tools with a component library system. The software process assumed by these tools may not take reuse into account. This is particularly true for tools that support embedded systems engineering, less so for object-oriented development tools.
Not-invented-here syndrome	Some software engineers prefer to rewrite components because they believe they can improve on them. This is partly to do with trust and partly to do with the fact that writing original software is seen as more challenging than reusing other people's software.

The reuse landscape

Summing up

- Although reuse is often simply thought of as the reuse of system components, there are many different approaches to reuse that may be used
- Reuse is possible at different levels
  - from ideas to design methods to models
  - from simple functions to complete application systems
- There exist many reuse techniques

Implementing software reuse



Approaches that support software reuse

Approach	Description
Application frameworks	Collections of abstract and concrete classes are adapted and extended to create application systems. e.g.,: CORBA, COM/DCOM, Java RMI, Eclipse Rich Client Platform
Application system integration	Two or more application systems are integrated to provide extended functionality e.g., see Configurable Application System in next slide
Architectural patterns	Standard software architectures that support common types of application system are used as the basis of applications (described in Chapters 6, 11 and 17 of the text book) e.g., SOA, Event-driven Architecture, Peer-to-peer, Microservices, Multitier Architecture (often Three-tier or N-tier), Model View Controller
Aspect-oriented software development	Shared components are woven into (interlaced with) an application at different places when the program is compiled. Described in web Chapter 31. e.g., AspectJ
Component-based software engineering	Systems are developed by integrating components (collections of objects) that conform to component-model standards. Described in Chapter 16. e.g., Enterprise Javabeans (EJB) Model, Component Object Model (COM) Model, .Net Model, Common Object Request Broker Architecture (CORBA) Component Model

Approaches that support software reuse

Approach	Description
Configurable application systems	Domain-specific systems are designed so that they can be configured to the needs of specific system customers. e.g., Tomcat, Microsoft Office, antivirus software, management software, ordering and invoicing, inventory management, and manufacturing scheduling
Design patterns	Generic abstractions that occur across applications are represented as design patterns showing abstract and concrete objects and interactions. A design pattern is a way of reusing abstract knowledge about a problem and its solution. A pattern is a description of the problem and the essence of its solution. Described in Chapter 7.
Enterprise Resource Planning (ERP) systems	Large-scale systems that encapsulate generic business functionality and rules are configured for an organization. e.g., Gazie, Gestionale Open, Invoicex, MosaicoX, Odoo (ex OpenERP), Opensuite ERP, Phasis, PMbyAS, Promogest, EBI Neutrino R1
Legacy system wrapping	Legacy systems (see Chapter 9) are "wrapped" by defining a set of interfaces and providing access to these legacy systems through these interfaces e.g., through Web services
Model-driven engineering	Software is represented as domain models and implementation independent models and code is generated from these models (described in Chapter 5)

Approaches that support software reuse

Approach	Description
Program generators	A generator system embeds knowledge of a type of application and is used to generate systems in that domain from a user-supplied system model.
Program libraries	Class and function libraries that implement commonly used abstractions are available for reuse.
Service-oriented Systems	Systems are developed by linking/composing shared services, which may be externally provided (described in Chapter 18)
Software Product Lines (SPLs)	An application type is generalized around a common architecture so that it can be adapted for different customers.
Systems of Systems	Two or more distributed systems are integrated to create a new system (described in Chapter 20)

Reuse planning factors

1. The development schedule for the software
   - A quick development might imply reusing entire off-the-shelf systems (large-grain reusable assets)… although the fit to requirements may be imperfect

2. The expected software lifetime
   - The development of long-lifetime systems (that require long-term maintenance and adaptation to changing requirements) might require reusing white-box components
   - Better avoid reusing black-box components for which suppliers may not be able to continue their support

3. The background, skills and experience of the development team
   - Effective reuse needs a lot of time and specific skills for understanding how to reuse
   - Better avoid reusing technologies for which the development team has no experience at all (unless you have enough time to learn)

Reuse planning factors

4. The criticality of the software and its non-functional requirements
   - Better avoid black-box reuse if the system has to be certified
   - Better avoid non domain-specific generator-based reuse techniques when, e.g., stringent performance requirements must be met (inefficient code might be produced)
   - Domain-specific code generator might be however inadequate

5. The application domain
   - Reuse as much as possible in those application domains where it is well known that mature configurable software can be profitably reused and certain qualities are guaranteed
   - Domain-specific code generator are welcome in this case

6. The execution platform for the software
   - Only platform-specific assets can be reused
   - Some components models, such as .NET, are specific to Microsoft platforms, and can be profitably reused only with Microsoft.

Application frameworks

Object-orientated approach

- Not easy reuse of objects (classes indeed) in different systems
  - Objects are often too small
  - Objects are often specialized for a particular application
  - Often it takes longer to understand and adapt the object than to re-implement it

- Object-oriented reuse is best supported in an object-oriented development process through larger-grain abstractions called application frameworks,
  - e.g., (historically) CORBA, COM/DCOM, Java RMI, Eclipse Rich Client Platform

Application frameworks

- Frameworks are moderately large entities that can be reused. They are somewhere between system and component reuse.
- Frameworks enable design reuse and specific classes reuse in that they are sub-system design made up of a collection of abstract and concrete classes and the interfaces between them.
- Frameworks are language-specific.
- The sub-system is implemented by adding components to fill in parts of the design and by instantiating the abstract classes in the framework.

Framework definition

- "... an integrated set of software artefacts (such as classes, objects and components) that collaborate to provide a reusable architecture for a family of related applications" (Schmidt et al. 2004)

- Frameworks provide support for generic features that are likely to be used in all applications of a similar type.

- For example, a user interface framework will provide support for interface event handling and will include a set of widgets that can be used to construct displays
  - e.g., popular framework for creating Web interface are Bootstrap. Foundation by ZURB, Semantic UI, Pure by Yahoo, UIkit by YOOtheme

Framework classes

System infrastructure frameworks

- Support the development of system infrastructures such as communications, user interfaces and compilers.

Middleware integration frameworks

- Standards and classes that support component communication and information exchange (e.g., EJB, .NET).

Enterprise application frameworks

- Support the development of specific types of application such as telecommunications or financial systems.

Web Application Frameworks (WAF)

- Web Application Frameworks (WAFs) are a more recent and very important type of framework.
  - Support the construction of dynamic websites as a front-end for web applications.

- WAFs are now available for all of the commonly used web programming languages, e.g., Java, Python, Ruby, etc.

- The architecture of a WAF, its Interaction model, is usually based on the Model-View-Controller composite pattern.

- Well known WAF are Apache Struts, Spring, Swing, Java Server Faces (Oracle ADF)

Top 10 Web Development Frameworks in 2019

Backend frameworks

- Express
- Django
- Rails
- Laravel
- Spring

Frontend Javascript Frameworks

- Angular
- React
- Vue
- Ember
- Backbone
- Final Word

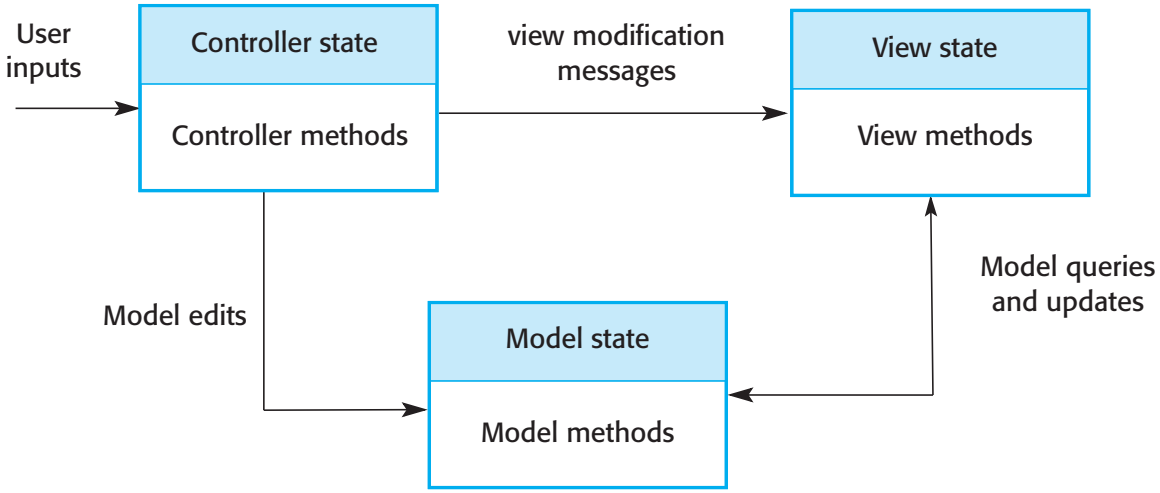
https://hackr.io/blog/top-10-web-development-frameworks-in-2019#Backend_frameworks

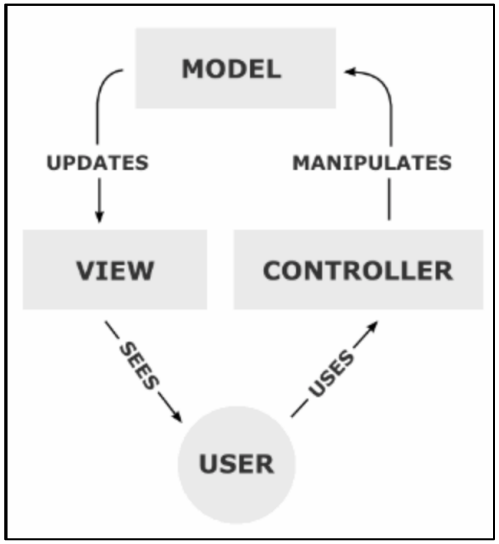
Model-View-Controller (MVC)

- Originally proposed as a system infrastructure framework for GUI design.
- It allows for the separation of the application state and the user interface to the application.
- An MVC framework supports the presentation of data in different ways and allows interaction with each of these presentations.

The MVC pattern

- When the data is modified through one of the presentations, the system model is changed and the controllers associated with each view update their presentation.





WAF features

Security

- WAFs may include classes to help implement user authentication (login) and access.

Dynamic web pages

- Classes are provided to help you define web page templates and to populate these dynamically from the system database.

Database support

- The framework may provide classes that provide an abstract interface to different databases.

Session management

- Classes to create and manage sessions (a number of interactions with the system by a user) are usually part of a WAF.

User interaction

- Most web frameworks now provide AJAX support (Holdener, 2008), which allows more interactive web pages to be created.

Extending frameworks

- Frameworks are generic and are extended to create a more specific application or sub-system. They provide a skeleton architecture for the system.

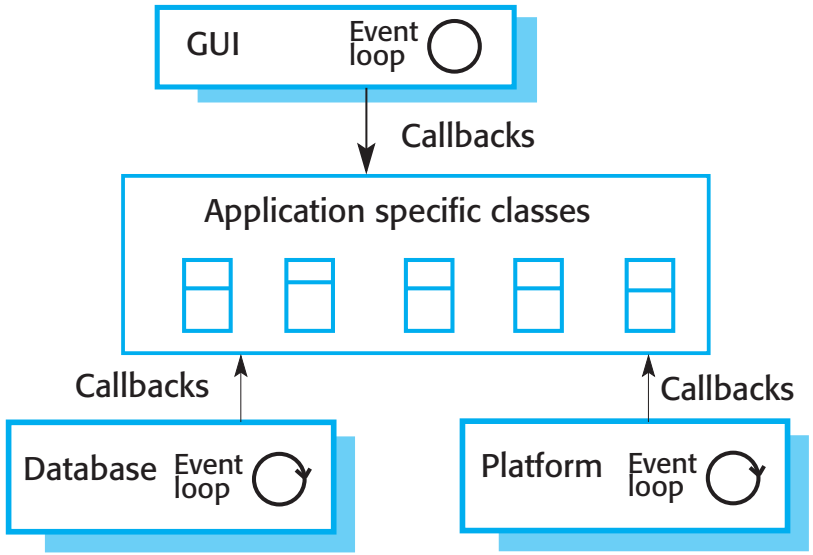
- Extending the framework involves
  - Adding concrete classes that inherit operations from abstract classes in the framework;
  - Adding methods (i.e., defining callbacks) that are called in response to events that are recognised by the framework.

- Problem with frameworks is their complexity which means that it takes a long time to use them effectively ☹

Inversion of control in frameworks

- Callbacks are methods that are called in response to events recognized by the framework
- Schmidt et al. 2004 call this "inversion of control"

In response to events from the user interface, database, platform, etc., these framework objects invoke 'hook methods' that are then linked to user-provided functionalities



The application-specific functionality responds to the event in an appropriate way

For example, a framework will have a method that handles a mouse click from the environment

This method calls the hook method, which you must configure to call the appropriate application methods to handle the mouse click

Software Product Lines

- Software Product Lines or Application Families are applications with generic functionality that can be adapted and configured for use in a specific context.

- A Software PL is a set of applications with a common architecture and shared components, with each application specialized to reflect different requirements.
  - For instance, a computer game product line.

- Adaptation may involve:
  - Component and system configuration;
  - Adding new components to the system;
  - Selecting from a library of existing components;
  - Modifying components to meet new requirements.

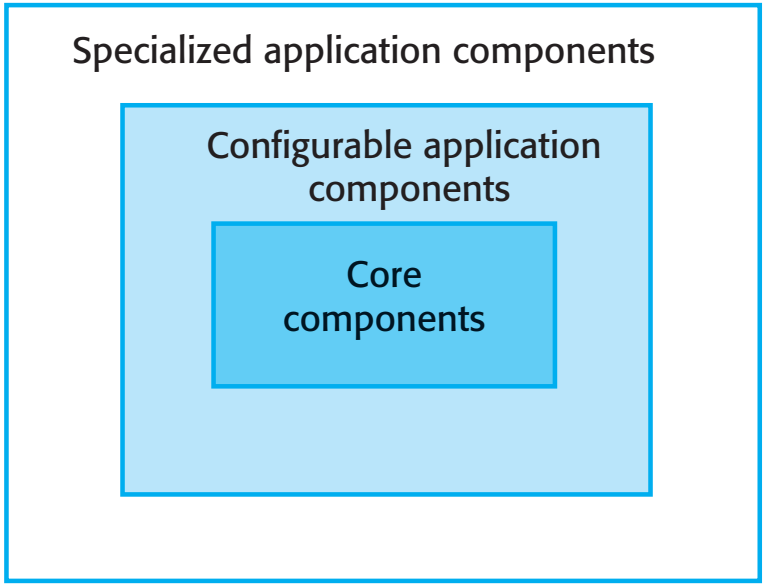
http://www.sei.cmu.edu/productlines/casestudies/catalog/index.cfm

Base components for a Software PLs

- Core components that provide infrastructure support. These are not usually modified when developing a new instance of the product line.

- Configurable components that may be modified and configured to specialize them to a new application. Sometimes, it is possible to reconfigure these components without changing their code by using a built-in component configuration language.

- Specialized, domain-specific components some or all of which may be replaced when a new instance of a product line is created.



Application frameworks and PLs

- Application frameworks rely on object-oriented features such as polymorphism to implement extensions. Product lines need not be object-oriented (e.g., embedded software for a mobile phone)

- Application frameworks focus on providing technical rather than domain-specific support. Product lines embed domain and platform information.

- Product lines are made up of a family of applications, usually owned by the same organization.

Product line architectures

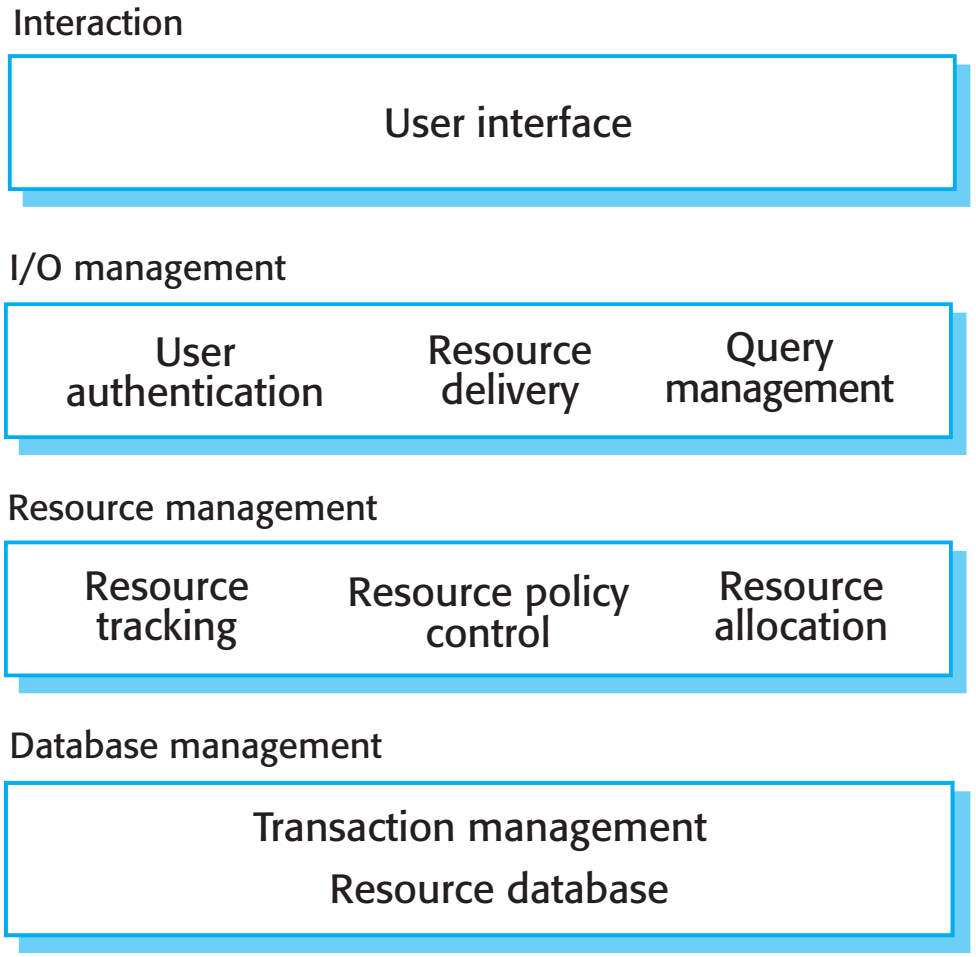
- Architectures must be structured in such a way to separate different sub-systems and to allow them to be modified.

- The architecture should also separate entities and their descriptions and the higher levels in the system access entities through descriptions rather than directly.

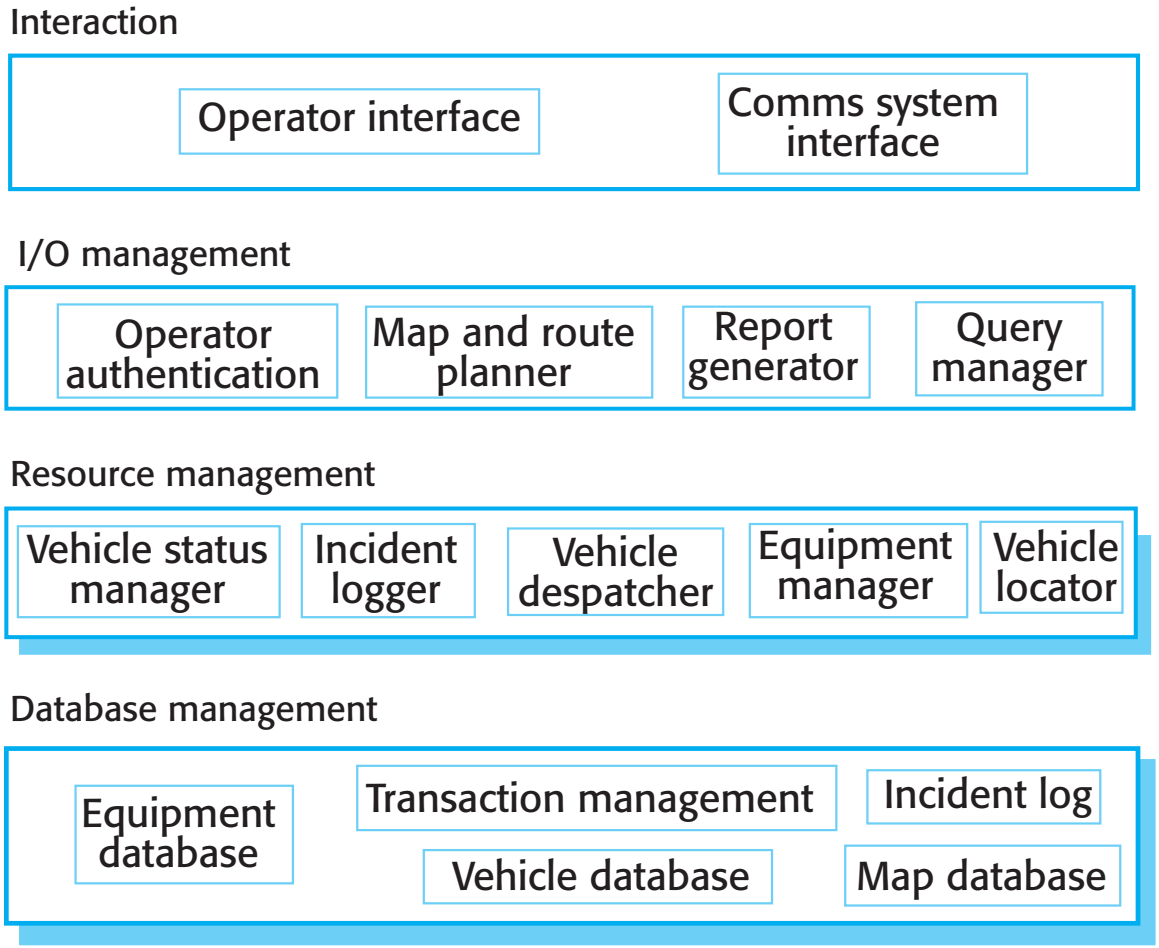
Vehicle dispatching

- A specialised resource management system where the aim is to allocate resources (vehicles) to handle incidents.
- Adaptations include:
  - At the UI level, there are components for operator display and communications;
  - At the I/O management level, there are components that handle authentication, reporting and route planning;
  - At the resource management level, there are components for vehicle location and despatch, managing vehicle status and incident logging;
  - The database includes equipment, vehicle and map databases.

The architecture of a resource allocation system



The product line architecture of a vehicle dispatcher



Product line specialisation

Platform specialization

- Different versions of the application are developed for different platforms.

Environment specialization

- Different versions of the application are created to handle different operating environments, e.g., different types of communication equipment.

Functional specialization

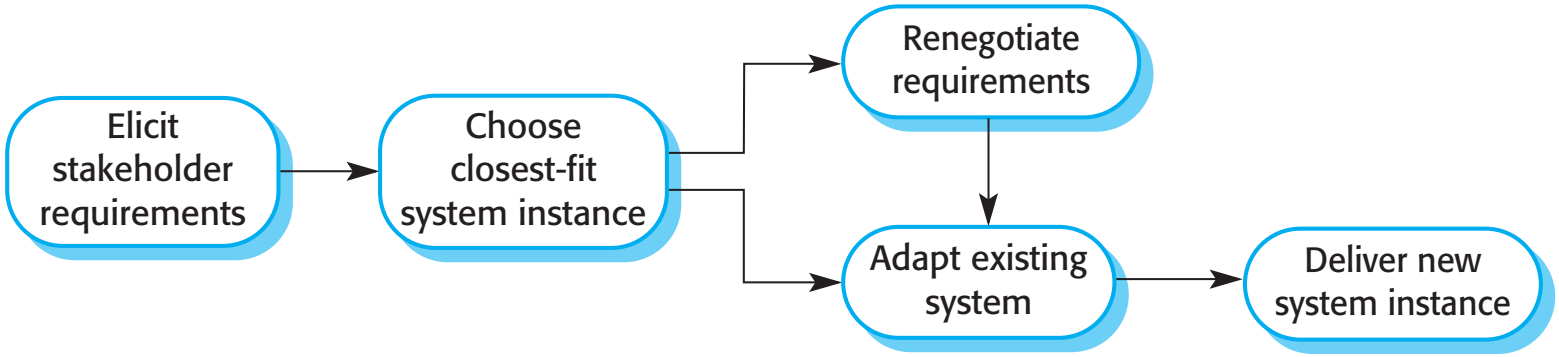
- Different versions of the application are created for customers with different requirements.

Process specialization

- Different versions of the application are created to support different business processes.

Product instance development

- Elicit stakeholder requirements
  - Use existing family member as a prototype
- Choose closest-fit family member
  - Find the family member that best meets the requirements
- Re-negotiate requirements
  - Adapt requirements as necessary to capabilities of the software
- Adapt existing system
  - Develop new modules and make changes for family member
- Deliver new family member
  - Document key features for further member development



Product line configuration

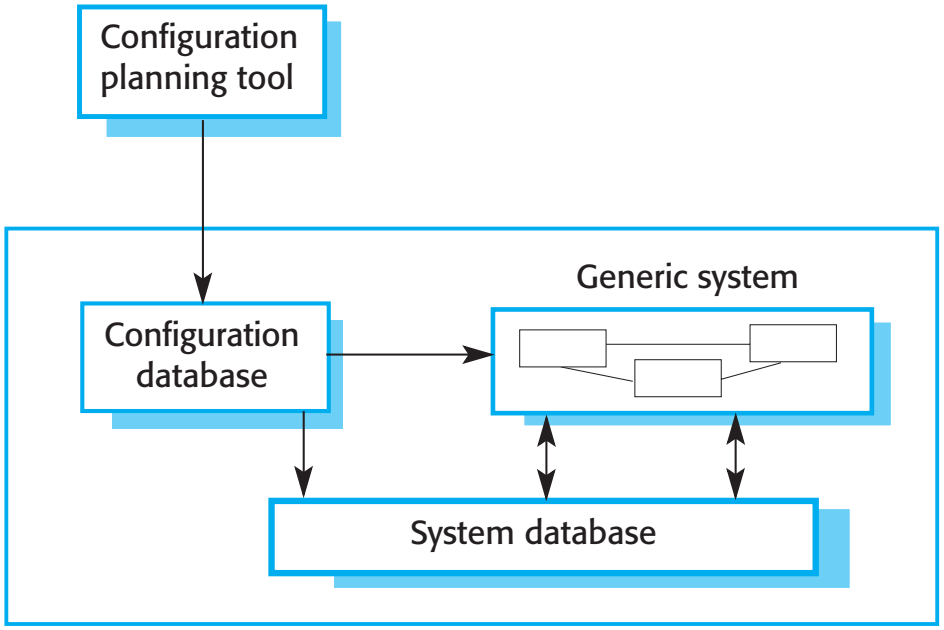
Design time configuration

- The organization that is developing the software modifies a common product line core by developing, selecting or adapting components to create a new system for a customer.

Deployment time configuration

- A generic system is designed for configuration by a customer or consultants working with the customer. Knowledge of the customer's specific requirements and the system's operating environment is embedded in configuration data that are used by the generic system.

Deployment-time configuration



Levels of deployment time configuration

- Component selection, where you select the modules in a system that provide the required functionality.

- Workflow and rule definition, where you define workflows (how information is processed, stage-by-stage) and validation rules that should apply to information entered by users or generated by the system.

- Parameter definition, where you specify the values of specific system parameters that reflect the instance of the application that you are creating.

Application System reuse

- A commercial-off-the-shelf (COTS) application system product is a software system that can be adapted for different customers without changing the source code of the system
  - e.g., Tomcat, Microsoft Office, antivirus software, management software, ordering and invoicing, inventory management, and manufacturing scheduling
- Application systems have generic features and so (often upon adaptation/customization) can be used/reused in different environments.
- Application system products are adapted by using built-in configuration mechanisms that allow the functionality of the system to be tailored to specific customer needs.
  - E.g., in a hospital patient record system, separate input forms and output reports might be defied for different types of patient.

Benefits of application system reuse

- As with other types of reuse, more rapid deployment of a reliable system may be possible.
- It is possible to see what functionality is provided by the applications and so it is easier to judge whether or not they are likely to be suitable.
- Some development risks are avoided by using existing software. However, this approach has its own risks, as I discuss below.
- Businesses can focus on their core activity without having to devote a lot of resources to IT systems development.
- As operating platforms evolve, technology updates may be simplified as these are the responsibility of the COTS product vendor rather than the customer.

Problems of application system reuse

- Requirements usually have to be adapted to reflect the functionality and mode of operation of the COTS product.
- The COTS product may be based on assumptions that are practically impossible to change.
- Choosing the right COTS system for an enterprise can be a difficult process, especially as many COTS products are not well documented.
- There may be a lack of local expertise to support systems development.
- The COTS product vendor controls system support and evolution.

Configurable application systems

- Configurable application systems are generic application systems that may be designed to support a particular business type, business activity or, sometimes, a complete business enterprise.
  - For example, an application system may be produced for dentists that handles appointments, dental records, patient recall, etc.

- Domain-specific systems, such as systems to support a business function (e.g. document management) provide functionality that is likely to be required by a range of potential users.

COTS-solution and COTS-integrated systems

Comparing two possible Application System Reuse approaches

Configurable application systems	Application system integration
Single product that provides the functionality required by a customer	Several heterogeneous system products are integrated to provide customized functionality
Based around a generic solution and standardized processes	Flexible solutions may be developed for customer processes
Development focus is on system configuration	Development focus is on system integration
System vendor is responsible for maintenance	System owner is responsible for maintenance
System vendor provides the platform for the system	System owner provides the platform for the system

ERP systems (configurable application systems)

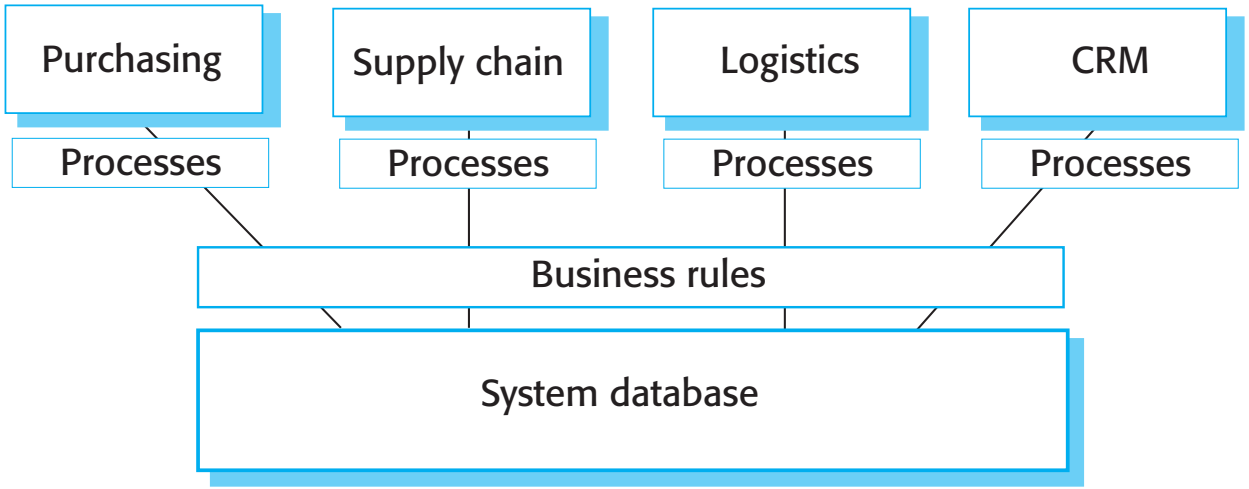
- An Enterprise Resource Planning (ERP) system is a generic system that supports common business processes such as ordering and invoicing, manufacturing, etc.

- These are very widely used in large companies - they represent probably the most common form of software reuse.

- The generic core is adapted by including modules and by incorporating knowledge of business processes and rules.

The architecture of an ERP system (a sample of)

- A number of modules to support different business functions.
- A defined set of business processes, associated with each module, which relate to activities in that module.
- A common database that maintains information about all related business functions.
- A set of business rules that apply to all data in the database.



ERP configuration

- Selecting the required functionality from the system.
- Establishing a data model that defines how the organization's data will be structured in the system database.
- Defining business rules that apply to that data.
- Defining the expected interactions with external systems.
- Designing the input forms and the output reports generated by the system.
- Designing new business processes that conform to the underlying process model supported by the system.
- Setting parameters that define how the system is deployed on its underlying platform.

Integrated application systems

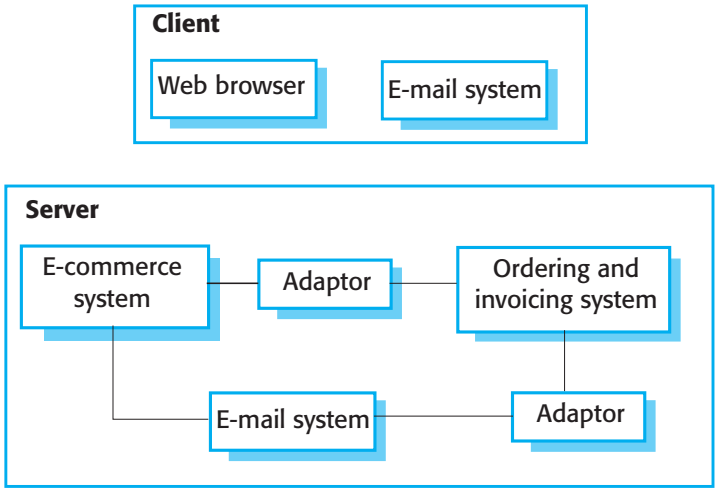
- Integrated application systems are applications that include two or more application system products and/or legacy application systems.
- You may use this approach when there is no single application system that meets all of your needs or when you wish to integrate a new application system with systems that you already use.

Design choices

- Which individual application systems offer the most appropriate functionality?
  - Typically, there will be several application system products available, which can be combined in different ways.
- How will data be exchanged?
  - Different products normally use unique data structures and formats. You have to write adaptors that convert from one representation to another.
- What features of a product will actually be used?
  - Individual application systems may include more functionality than you need and functionality may be duplicated across different products.

An integrated procurement system

- Sample COTS integration: the goal is to develop a procurement system that allows staff to place orders from their desk.



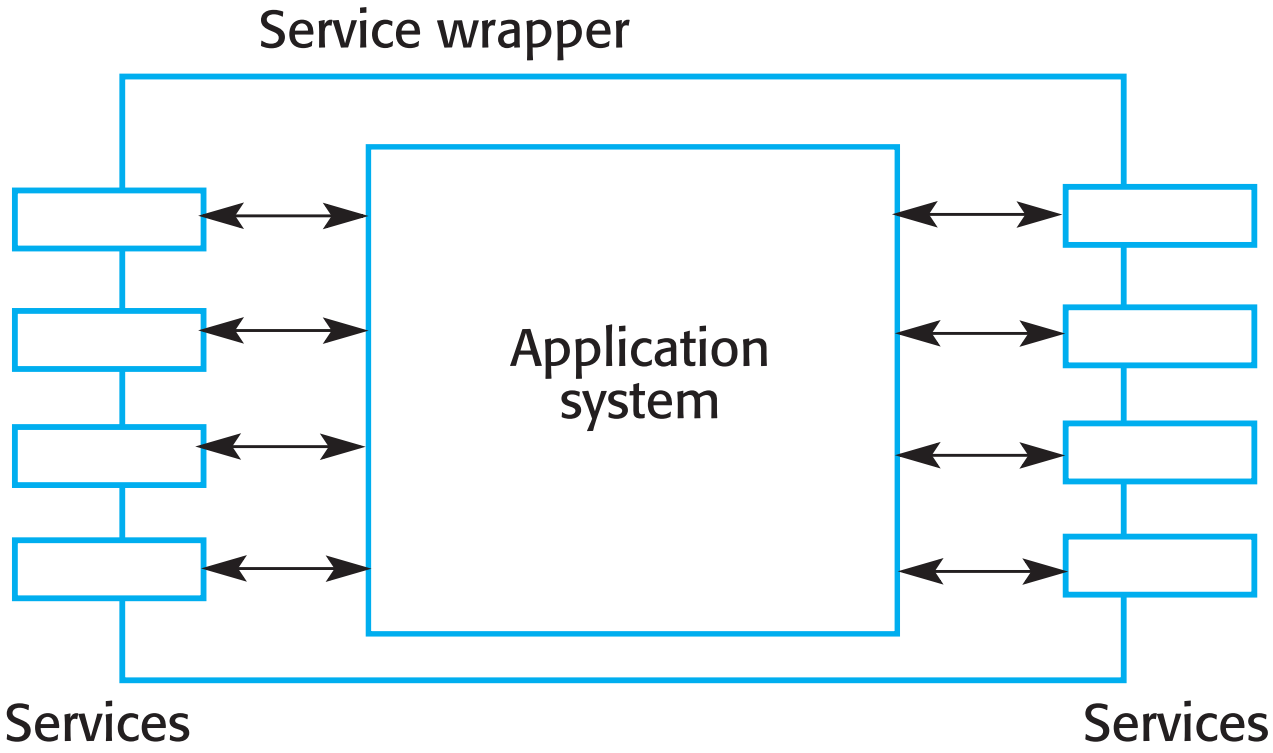
Service-oriented interfaces

- Application system integration can be simplified if a service-oriented approach is used.

- A service-oriented approach means allowing access to the application system's functionality through a standard service interface, with a service for each discrete unit of functionality.

- Some applications may offer a service interface but, sometimes, this service interface has to be implemented by the system integrator. You may have to program a wrapper that hides the application and provides externally visible services (next slide...).

Application wrapping



Application system integration problems

- Lack of control over functionality and performance
  - Application systems may be less effective than they appear
- Problems with application system interoperability
  - Different application systems may make different assumptions that means integration is difficult
- No control over system evolution
  - Application system vendors control evolution, not system users
- Support from system vendors
  - Application system vendors may not offer support over the lifetime of the product

Key points

- There are many different ways to reuse software. These range from the reuse of classes and methods in libraries to the reuse of complete application systems.

- The advantages of software reuse are lower costs, faster software development and lower risks. System dependability is increased. Specialists can be used more effectively by concentrating their expertise on the design of reusable components.

- Application frameworks are collections of concrete and abstract objects that are designed for reuse through specialization and the addition of new objects. They usually incorporate good design practice through design patterns.

Key points

- Software product lines are related applications that are developed from one or more base applications. A generic system is adapted and specialized to meet specific requirements for functionality, target platform or operational configuration.

- Application system reuse is concerned with the reuse of large-scale, off-the-shelf systems. These provide a lot of functionality and their reuse can radically reduce costs and development time. Systems may be developed by configuring a single, generic application system or by integrating two or more application systems.

- Potential problems with application system reuse include lack of control over functionality and performance, lack of control over system evolution, the need for support from external vendors and difficulties in ensuring that systems can inter-operate.